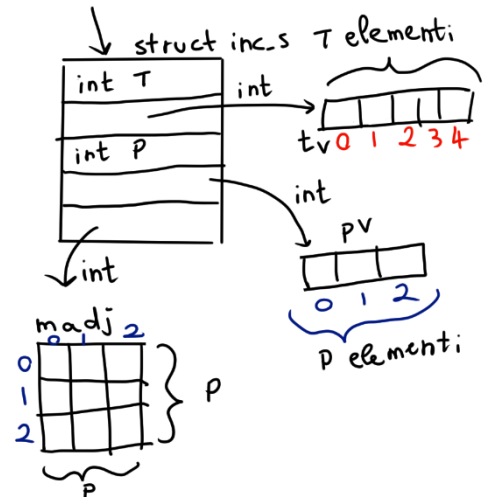


Esercizio da 18 punti

Strutture dati

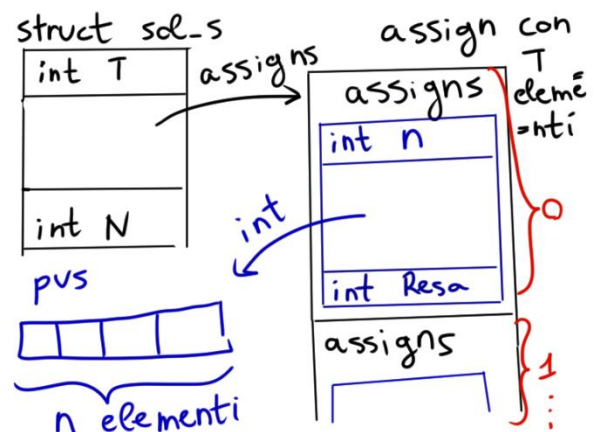
E' stato creato una struttura che contiene tutte le informazioni ottenute dal file input.txt, esso possiede il numero di incarichi e il vettore di incarichi, il numero di persone e il vettore esperienza delle persone dove ogni persona viene indentificata con un indice, e la matrice di sinergia.



Una struttura per contenere la soluzione composto dal numero degli incarichi, il numero di persone totali presenti e un vettore di tipo assigns, ogni indice di assign rappresenta l'incarico con lo stesso indice nella struct `inc_s` (in rosso nel disegno).

Ogni "casella" di assign contiene:

- il numero di persone incaricate alla mansione i-esima
- il vettore di indici che rappresenta quali sono le persone incaricate
- la resa del singolo incarico



Scelte algoritmiche

Acquisizione dati – getproposta

L'input della proposta è stato pensato nel formato:

- La prima riga con due interi: T (numero di incarichi) N (numero di persone occupate)

A seguire T righe ognuna contenente

- Numero di persone assegnate all'incarico i-esimo con accanto la lista delle persone rappresentato come indici di interi

Nell'aquisizione della proposta la resa per incarico viene inizializzata a zero.

Controllo – funzione check

Se c'è lo stesso numero totale di incarichi e se non ci sono persone in più di quanto siano realmente disponibili (controllo che il file proposta sia corretto e corrispondente al file input) oppure che non ci siano persone che non fanno niente (input con contenuti sensati).

Controllo che ogni singola persona stia svolgendo una singola mansione usando un vettore di flag parallelo al vettore $G \rightarrow pv$ delle risorse umane inizialmente azzerato, ogni volta che trovo un lavoratore occupato in una mansione aggrando a 1 il valore all'indice corrispondente nel flag, se trovo un lavoratore che è già impiegato in più mansioni ritorna 0.

Per ogni mansione se la resa ha superato il 75% viene scelto il minimo tra la resa limite e la resa reale altrimenti viene azzerata, infine ne faccio la sommatoria.

In sede d'esame è stato considerato solo la sinergia tra lavoratori adiacenti che lavorano sulla stessa mansione: se i lavoratori di indice 0 2 e 5 lavorano sullo stesso task il programma scritto considera solo la sinergia tra 0-2 e 2-5 senza considerare quella tra 0-5. Valore che invece andrebbe aggiunto alla sommatoria. Per mantenersi il più possibile vicino alla strategia iniziale nel codice che verrà consegnato non è stata corretta questa parte anche se sarebbe errata.

Ottimizzazione – funzione solve e solve_r

In solve_r sono state usate le disposizioni ripetute nella funzione ricorsiva, dove per ogni lavoratore posso scegliere di incaricarlo a una certa mansione.

E' una funzione void con unica condizione di ritorno l'aver scorso tutto il vettore pv (quando pos, il contatore, raggiunge il valore k pari a $G \rightarrow P$, numero di addetti al lavoro) dopo aver calcolato in terminazione il punteggio di una certa disposizione di lavoratori in certe mansioni, se il punteggio è maggiore del migliore stimato si aggiorna il valore migliore e si copia la soluzione in rr (vettore di soluzioni). In scelta viene salvato l'indice dell'incarico da svolgere in tmpr[i] (la casella i-esima del vettore della soluzione temporanea) la scelta è tra i possibili incarichi. Non è presente pruning in scelta.

Solve è il wrapper per la funzione ricorsiva, in essa ci stanno le allocazioni e le inizializzazioni.

Inizialmente si è pensato di usare la funzione check (soluzione alla richiesta del secondo punto) per il calcolo del punteggio in terminazione della ricorsione, per questo motivo è stato iniziato l'allocazione di ris ma è stato preferito creare una funzione per il calcolo della resa con una struttura dati più semplice, ovvero un vettore che contiene al suo interno il numero dell'incarico per ogni lavoratore, di quale lavoratore si tratta lo si identifica tramite l'indice.

Inoltre sono state usate le funzioni:

- Calcolaresa: Calcola la resa totale
 - Esso usa il vettore di interi chiamata sol (tmpr in solve_r) e non la struct sol_s

- Vengono azzerate tramite calloc due vettori, uno per la resa del i-esimo incarico e uno che funziona come contenitore temporaneo degli indici dei lavoratori assegnati all'i-esimo incarico.
- Per ogni lavoratore aggiungo alle rese dell'incarico assegnato la sua esperienza
- Originariamente dovrebbe esserci stato un altro ciclo a sé stante con i che parte da zero e aumenta fino al numero di incarichi dove per ogni incarico c'è un ciclo interno (j) dove si salvano gli indici del personale incaricato e si sommano le loro sinergie, questa era l'idea ma è stato scritto negli ultimi minuti e non rappresenta cosa farebbe il codice, invece:
 - Il contributo dovuto alle sinergie è stato contato in parte dentro al ciclo di sommatoria delle esperienze e in parte fuori da esso (la parte che non è stata completata)
 - Mentre si stavano sommando le esperienze per ogni singolo incarico, se l'elemento all'indice i della soluzione vale 0 (appartiene alla prima task) si salva l'indice (che rappresenta il lavoratore) in un vettore temporaneo vectsin, esso rappresenta gli impiegati che stanno lavorando sull'incarico corrente (in questo caso incarico 0), dopo aver ciclato su ogni persona si somma alla resa[0] le sinergie, quest'ultime sono tra due persone, quindi con n persone dovrò contare n-1 sinergie, decidendo di usare la sinergia tra il lavoratore corrente e quello subito precedente che lavora sulla stessa task inizio il ciclo in s da 1 fino a k.
 - La seconda parte è come l'idea originaria ma con indice che parte da 1 in quanto la resa[0] è stata già considerata, internamente ha lo stesso meccanismo di quello usato per il l'incarico zero ma in un ciclo per incarico i-esimo crescente
- Copyvect : Copia il contenuto di un vettore di n elementi in un altro vettore sempre di n elementi

Differenze

- Aggiunto main, librerie standard, funzioni di stampa
- Aggiunta una funzione per calcolare il minimo tra due valori
- Sostituite le virgole con ; nelle definizioni delle struct
- Aggiunta deallocazione di tutte le strutture dati
- In malloc2d, come da sintassi corretta è stato:
 - Modificato elemento ritornato alla funzione chiamante da int a **int
 - Reso malloc interno al ciclo da *mat a mat[i]
- In leggibile
 - Nella fretta mentre si stava facendo la malloc di G->pv si stava già pensando al passo successivo (fscanf) si è finito per usare la malloc ma con la sintassi

necessaria per la lettura da file dentro la parentesi, sostituito con la sintassi corretta

- In getproposta sono stati compiuti vari errori di distrazione:
 - scambiato il tipo (sol) con il la variabile (ris) nella malloc di assign
 - `ris->assign[i]->pvs` è sostituito con `ris->assign[i].pvs` nel malloc di `ris->assign[i].pvs`
 - `assign[i].h` sostituito con `assign[i].n`
 - `ris->assign[i].pvs[i]` sostituito con `ris->assign[i].pvs[j]` dentro il for con la j
- In check
 - Aggiunta la chiamata alla funzione che seleziona il minimo tra il valore dato dalla somma tra le esperienze dei lavoratori e le sinergie tra di loro e il valore dell'incarico, come da richiesta
 - Errato passaggio del parametro G, dovrebbe essere inc al posto di int come da intenzioni originarie
 - Aggiunta parentesi] in `flag[ris->assign[i].pvs[j]]`
 - Aggiunta la chiusura di parentesi graffa (in particolare andava messa dopo il ciclo interno con la j , prima del controllo relativo al 75% di resa) in quanto il confronto andrebbe fatto una volta esaminati tutti i lavoratori per una mansione, dopo che il for interno sia terminato, una volta per mansione
- In solve_r
 - Typo nel nome della funzione `calcolaresa()`
 - Tolto il passaggio del vettore flag inutilizzato
- In solve
 - Aggiunto `.pvs` dopo `ris->assign[i]`
 - Aggiunta stampa della soluzione
 - Tolto il vettore flag e ris (variabile di tipo sol) inutilizzato e non è stato sviluppato la parte relativa al salvataggio in ris e la sua riallocazione
- In calcolaresa
 - Da `vectsin[i]=j` a `vectsin[k]=j`: indice sostituito da i a k
 - Sommatoria con il minimo tra la resa reale di una certa task e il valore stabilito della task come da richiesta
 - Nella somma del contributo dovuto alla sinergia nel calcolo della resa sostituito k con s dentro `sin[vectsin[k-1]][vectsin[k]]` per poter scorrere il vettore sommando il valore delle sinergie ad indice crescente invece di sommare un valore fisso

Commenti

Si noti che sia nel controllo che nella ricerca della soluzione ottima non è presente il vincolo di assegnare almeno una persona a ogni task, infatti nel controllo il numero di T rappresenta il numero di task totali e non il numero di task con persone assegnate, mentre nella ricerca nel caso la stima migliore fosse assegnando 0 persone a una task esso viene salvato, si riflette anche nella funzione di controllo ad-hoc usando i vettori creati nella parte di ottimizzazione.

Non si ha avuto abbastanza tempo per terminare la funzione di conteggio dei punti. La parte mancante ha la stessa struttura di quella per calcolare la resa[0] ma dentro un for crescente che parte da 1, dove viene calcolata la resa i-esima in posizione i del vettore delle rese per task.

Nel codice funzionante sono stati eliminati i commenti relativi al metodo di risoluzione sostituiti con le modifiche apportate.